Control-M Ingestion — the `ingest-controlm` chain

Rev 2 · 2026-07-07 — reflects commit `107581d` (enforced load order + `:ControlMApplication` in the folder pass) · **Classification** Internal · **Companion** `docs/controlm-staging-ingestion-flow.md` §3a (load-order contract).

> **WHAT CHANGED IN REV 2**
> **1.** The chain order is now **contractual** (a test enforces it), framed as standard Neo4j import discipline — see §3b. **2.** The **folder pass now derives two grouping labels**, not one: `DATA_CENTER → :ControlMServer` *and* header-row `APPLICATION → :ControlMApplication` (+ `CONTAINS_FOLDER`). See §2, §4, §6.

```
poetry run drydocs ingest-controlm --use-oracle --folder 'PRARAG-HLDM-85025-PEX%'
```

Reminder: `--folder` is a **bind variable**, not interpolated (`cursor.execute(query, binds)`, `oracle_adapter.py:61`); `%` is a bound `LIKE` wildcard, injection-safe.

1 · THE FOUR-LAYER FRAME

Layer	This chain's contribution	Artifact
1 · Taxonomy	`ControlMApplication`, `ControlMServer > ControlMFolder > ControlMJob`, `Condition`	`config/taxonomy/controlm.yaml`
2 · Ontology	PROV binding per edge, HITL-confirmed	`config/taxonomy-ontology-map.yaml`
3 · Knowledge graph	populated Neo4j nodes + edges	Neo4j
4 · Context graph	— (future)	—

Rule: import as taxonomy (classification) → apply the confirmed ontology rule → then load.

NOTES

2 · SOURCE SYSTEM — THE PSGMGR CM_ REPLICA

Read-only replica (`psgmgr`, grantee `CM_RO_USER`). Four objects — and, new in Rev 2, a **second read of `CM_DEF_VJOB`** (the folder header row):

#	Concept	Vendor (6.4.01)	psgmgr replica	Grain / use
1	Folder / schedule table	`CMS_SCHEDT`	`CM_DEF_VTAB`	one row / folder
2	Job definition	`CMS_JOBDEF`	`CM_DEF_VJOB`	one row / (folder, job, version)
2b	**Folder header row**	SMART-table hdr	`CM_DEF_VJOB JOB_ID = 1`	**folder-grain `APPLICATION`** — LEFT-JOINed into the folder extract
3	IN-conditions	`CMS_CON_J` (in)	`CM_DEF_LNKI_P_VW`	one row / (job, condition)
4	OUT-conditions	`CMS_CON_J` (out)	`CM_DEF_LNKO_P_VW`	one row / (job, condition)

Filters & join keys

Item	Detail
`IS_CURRENT_VERSION = '1'`	**String** (`VARCHAR2(1)`) — jobs, conditions, **and the header-row join**. Not on folders (no version column).
`USER_DAILY IS NOT NULL`	the only "actively scheduled" gate on a folder.
Job ↔ folder	`CM_DEF_VJOB.TABLE_ID = CM_DEF_VTAB.TABLE_ID`
Header ↔ folder (new)	`CM_DEF_VJOB.(TABLE_ID, JOB_ID=1, IS_CURRENT_VERSION='1')` — LEFT JOIN
Condition ↔ job	`CM_DEF_LNK{I,O}_P_VW.(TABLE_ID, JOB_ID, VERSION_SERIAL)`
Dependency (derived)	`LNKI(B).CONDITION = LNKO(A).CONDITION` ⇒ B depends on A

> **THREE DIFFERENT "APPLICATIONS" — KEEP THEM APART**
> header-row `:ControlMApplication` (a Control-M grouping) ≠ SEAL `:Application` (business app, `seal_id`) ≠ the folder-name `app_code` (positions 3–5). Also: folder name = `SCHED_TABLE`, not the denormalized `PARENT_TABLE` on the job.

NOTES

3 · ER DIAGRAM & THE ORDER THE TABLES ARE USED

3a · Source ER (incl. the header-row LEFT JOIN)

```
       CM_DEF_VTAB  (folder)                   PK: TABLE_ID
       |  SCHED_TABLE (name), DATA_CENTER,       filter: USER_DAILY IS NOT NULL
       |  USER_DAILY, TABLE_STATUS
       |
       |  LEFT JOIN CM_DEF_VJOB H  ON H.TABLE_ID=TABLE_ID AND H.JOB_ID=1   <== NEW
       |            (SMART-table header row -> H.APPLICATION)   IS_CURRENT_VERSION='1'
       |
       |  TABLE_ID (1)---------------(many)
       v
       CM_DEF_VJOB  (job)                      PK: (TABLE_ID, JOB_ID, VERSION_SERIAL)
       |  JOB_NAME, APPLICATION, OWNER, AUTHOR,  filter: IS_CURRENT_VERSION='1'
       |  NODE_ID, CMD_LINE, CYCLIC
       |
   +---+----+  (TABLE_ID, JOB_ID, VERSION_SERIAL)
   v        v
 LNKI_P_VW  LNKO_P_VW                          filter: IS_CURRENT_VERSION='1'
 (IN cond)  (OUT cond, SIGN +/-)
     +---- derived: LNKI.CONDITION = LNKO.CONDITION ----+
```

3b · Load order — now a CONTRACT (`test_ingest_chain_order_is_enforced`)

Standard Neo4j import discipline: constraints first; all nodes before any relationships; relationship passes `MATCH` endpoints (never `MERGE`), so a missing endpoint surfaces instead of a ghost node.

#	Pass	Nodes MERGEd	Relationships	Endpoint rule
0	`constraints.cypher` (bootstrap)	—	—	every key below is constraint-backed
1	**folders**	`:ControlMFolder` + **two grouping labels**: `DATA_CENTER→:ControlMServer`, header `APPLICATION→:ControlMApplication`	`SCHEDULED_ON`, `CONTAINS_FOLDER`	self-contained — all endpoints created here
2	**jobs**	`:ControlMJob`	`CONTAINS_JOB`	MATCH folder (from pass 1)
3	**conditions in / out**	`:Condition` (shared `(folder_id,name)`)	`REQUIRES_IN` / `EMITS_OUT`	MATCH job `(folder_id, job_id)`
4	**dependencies** (separate, edge-only)	*none*	`WAS_INFORMED_BY`	MATCH **both** jobs — never creates nodes
—	SEAL attribution (K2, **gated**)	*none*	`WAS_ASSOCIATED_WITH`	only after jobs *and* `:Application` exist

Passes 3–5 run unless `--skip-part2`. Both grouping nodes exist before jobs. `m3-verify` gained a no-orphan-`:ControlMApplication` check.

3c · Target graph model (Rev 2)

```
(:ControlMApplication:Collection {name})          <== NEW (header-row APPLICATION)
       |  CONTAINS_FOLDER                              (prov:hadMember)
       v
(:ControlMServer:Platform {name})
       ^  SCHEDULED_ON {since}                        (no PROV - infra)
       |
(:ControlMFolder:Collection {folder_id, sched_table, env, lob, app_code, folder_type, active})
       |  CONTAINS_JOB                                 (prov:hadMember)
       v
(:ControlMJob:Activity {folder_id, job_id, job_name, application, owner, author, cmd_line, ...})
       |  REQUIRES_IN_CONDITION {odate, and_or, isn}       (prov:used)
       |  EMITS_OUT_CONDITION   {odate, sign, isn}         (prov:generated)
       v
(:Condition:Entity {folder_id, name, version_serial})

(:ControlMJob) --WAS_INFORMED_BY {via_condition, recursion_level, dependency_path}--> (:ControlMJob)
every node    --WAS_GENERATED_BY {source:'BMC'}--> (:JobRun {run_id, kind:'load'})
```

NOTES

4 · FIELD → LABEL / NODE / EDGE MAPPING

Stage 1 — folders → Folder + Server + ControlMApplication

Source `CM_DEF_VTAB` **LEFT JOIN** `CM_DEF_VJOB` header row.

SQL column	Node property	Notes
`T.TABLE_ID`	`ControlMFolder.folder_id`	node key (UNIQUE)
`T.SCHED_TABLE`	`.sched_table`	authoritative folder name
`T.DATA_CENTER`	`ControlMServer.name`	P12/P14/P32/P33
`H.APPLICATION`	`ControlMApplication.name`	**NEW** — header row (`JOB_ID=1`); may be NULL
`T.USER_DAILY`	`.user_daily`, `.active`	`active = not null/empty`
parsed SCHED_TABLE	`.environment/.lob/.app_code/.folder_type`	via `folder_name.py`

Edges: `(:ControlMFolder)-[:SCHEDULED_ON {since}]->(:ControlMServer)` ; **`(:ControlMApplication)-[:CONTAINS_FOLDER]->(:ControlMFolder)`** — only when `row.application` non-null (`WHERE` guard: the null-key-MERGE safeguard). Header-less folders load without an application node.

Stage 2 — jobs → `:ControlMJob:Activity`

Node key `(folder_id, job_id)` (JOB_ID folder-scoped); `VERSION_SERIAL` = audit property.

SQL column	→ property
`TABLE_ID`, `JOB_ID`	`.folder_id`, `.job_id` (key; folder MATCHed)
`JOB_NAME`, `APPLICATION`, `GROUP_NAME`, `TASK_TYPE`	`.job_name`, `.application` (≠ SEAL), `.group_name`, `.task_type`
`OWNER`, `AUTHOR`, `NODE_ID`, `CMD_LINE`	`.owner`, `.author`, `.node_id`, `.cmd_line`
`CYCLIC*`, `IS_CURRENT_VERSION`, `VERSION_*`	props; `.active = IS_CURRENT_VERSION='1'`

Edge: `(:ControlMFolder)-[:CONTAINS_JOB]->(:ControlMJob)`.

Stages 3 & 4 — conditions → `:Condition:Entity` · Pass 4 — dependencies (edge-only)

Stage	Edge (job → condition)
3 IN (`LNKI_P_VW`)	`REQUIRES_IN_CONDITION {odate, and_or, parentheses, isn}`
4 OUT (`LNKO_P_VW`)	`EMITS_OUT_CONDITION {odate, sign, isn}`

Dependencies is a **pure edge pass**: MERGEs no nodes, MATCHes both jobs `(folder_id, job_id)`, dir successor→predecessor; `recursion_level` / `dependency_path` ON CREATE only; key `via_condition`.

Identity & provenance

Node	Key constraint
`:ControlMApplication`	`name` UNIQUE — **new** (`controlmapplication_name`; EXPECTED 37→38)
`:ControlMServer`	`name` UNIQUE
`:ControlMFolder`	`folder_id` UNIQUE
`:ControlMJob`	`(folder_id, job_id)` NODE KEY
`:Condition`	`(folder_id, name)` NODE KEY

Every touched node also gets `-[:WAS_GENERATED_BY {source:'BMC'}]->(:JobRun)` (plan 06 proposes to demote this blanket edge).

NOTES

5 · TAXONOMY (LAYER 1)

`config/taxonomy/controlm.yaml` — **pure classification** (`classification_only: true`; `authority: bmc-baseline`; `classification: Internal`). Hierarchy `ControlMServer > ControlMFolder > ControlMJob`; `Condition` keyed `(folder_id, name)`. The `controlm-q1q3-phase1` gate (2026-07-07) added **`ControlMApplication`** as a folder grouping (Control-M `APPLICATION`, *not* the SEAL business app).

Sample example folders: **161015** `PRARAG-HLDM-85025-PEX-TRUST-DLY` (P12) & **161016** `PRARAG-HLDM-85025-PEX-TRUST-CYC` (P14).

6 · ONTOLOGY (LAYER 2)

`config/taxonomy-ontology-map.yaml` — HITL-confirmed bridge. Lifecycle `proposed → confirmed → applied`.

Edge (Neo4j)	From → To (PROV)	Matrix row	PROV term	Status
`CONTAINS_FOLDER`	Collection → Collection	Collection → any	`prov:hadMember`	**applied (Rev 2)**
`CONTAINS_JOB`	Collection → Activity	Collection → any	`prov:hadMember`	applied
`SCHEDULED_ON`	Collection → Platform	infra (no PROV)	—	confirmed
`REQUIRES_IN_CONDITION`	Activity → Entity	Activity → Entity	`prov:used`	confirmed
`EMITS_OUT_CONDITION`	Activity → Entity	Activity → Entity	`prov:generated`	confirmed
`WAS_INFORMED_BY`	Activity → Activity	Activity → Activity	`prov:wasInformedBy`	confirmed
`WAS_GENERATED_BY`	Entity/Activity → JobRun	—	`prov:wasGeneratedBy`	applied

`CONTAINS_FOLDER` reuses vocab `m3_contains_folder` (flipped **planned** → **active** this gate); the loader that lands it is the **folder pass** (`controlm_folders.cypher`), not jobs. Label duals encode PROV type: `:ControlMApplication:Collection`, `:ControlMFolder:Collection`, `:ControlMJob:Activity`, `:Condition:Entity`, `:JobRun`. `ControlMServer` is a local **Platform**, deliberately not a `prov:Agent`.

Not yet live: `WAS_ASSOCIATED_WITH` job→SEAL app (proposed, K2); `OBSERVES` job-run SOSA observation (proposed, gate not run).

NOTES

7 · WORKED EXAMPLE — `--FOLDER 'PRARAG-HLDM-85025-PEX%'`

7a · What the pattern matches

`SCHED_TABLE LIKE 'PRARAG-HLDM-85025-PEX%'` is a **prefix** match → both 85025 folders: `161015` (…-DLY, P12) and `161016` (…-CYC, P14). Not `161014` (…-**111027**-…). Drop the `%` for a single exact folder.

7b · Folder-name parse

Pos	Char	Meaning
1	P	environment = **Production**
2	R	LOB = **Retail**
3–5	ARA	app_code = **ARA**
6	G	folder_type = **Smart folder**

> **TRAP.**
> Folder *type* = prefix pos 6 (`G`), not the `DLY` / `CYC` suffix. And this `app_code=ARA` is a **third** thing, separate from the header-row `:ControlMApplication` (§7d).

7c · The five SQL steps (one shared bind dict)

```
binds = {"folder_filter": "PRARAG-HLDM-85025-PEX%", "run_as": None, "developer_sid": None, "row_cap": None}
```

```
-- Step 1 · folders  (Rev 2: LEFT JOIN the header row for APPLICATION)
SELECT T.TABLE_ID AS folder_id, T.SCHED_TABLE AS sched_table, T.DATA_CENTER AS data_center,
       H.APPLICATION AS application, T.USER_DAILY, ...
FROM   psgmgr.CM_DEF_VTAB T
LEFT JOIN psgmgr.CM_DEF_VJOB H
       ON  H.TABLE_ID = T.TABLE_ID
       AND H.JOB_ID   = 1                -- SMART-Table header row
       AND H.IS_CURRENT_VERSION = '1'
WHERE  T.USER_DAILY IS NOT NULL
  AND  T.SCHED_TABLE LIKE :folder_filter;  -- -> 161015, 161016

-- Steps 2-5 (jobs, IN/OUT conditions, recursive dependencies): UNCHANGED this commit.
--   each joins CM_DEF_VTAB and filters SCHED_TABLE LIKE :folder_filter
--   step 5 anchors on JOB_DEF.PARENT_TABLE LIKE :folder_filter, then recurses across folders.
```

7d · What lands in the graph (from the sample)

1. **2 folders** (`161015`, `161016`) → `SCHEDULED_ON` P12, P14; both parse env=Production, lob=Retail, app_code=ARA.
2. **:ControlMApplication (Rev 2):** under `--use-oracle`, each folder's header row (`JOB_ID=1`) supplies `APPLICATION` → a `:ControlMApplication {name}` node + `CONTAINS_FOLDER` to the folder. *(CSV sample mode without header rows: `application` NULL → `WHERE` guard skips it; folders still load.)*
3. **8 jobs** (5 in 161015 + 3 in 161016), each `CONTAINS_JOB`.
4. **Condition nodes** (5 DLY + 3 CYC names) wired by `REQUIRES_IN` / `EMITS_OUT`, shared where names match.
5. **WAS_INFORMED_BY** among ARA jobs where IN-cond = another's OUT-cond (e.g. `…_PLCT → …_PREPROC → …_FW`).
6. Every node gets one `WAS_GENERATED_BY` → this run's `:JobRun`.

NOTES

7e · Execution path (code)

```
cli.ingest_controlm(use_oracle=True, folder='PRARAG-HLDM-85025-PEX%')
  -> _gate_source("controlm-psgmgr")                    # D3 confirmed-gate
  -> scope  = _scope_binds(folder, None, None, None)
  -> stages = [folders, jobs, cond_in, cond_out, deps]  # order IS the contract
  -> for stage in stages:
       sql     = (SQL_DIR / stage.sql).read_text()      # static file, verbatim
       adapter = OracleAdapter(query=sql, bind_params=scope)
       cursor.execute(sql, scope)                       # <- BIND, not string-format
       rows -> Loader.load() -> UNWIND $batch ... MERGE  # the .cypher for that stage
```

APPENDIX — STICKY-NOTE GOTCHAS

1. Load order is **contractual** (`test_ingest_chain_order_is_enforced`): constraints → folders → jobs → conditions → **deps (edge-only)** → K2 (gated).
2. Folder pass now makes **two** grouping nodes: `:ControlMServer` (DATA_CENTER) and `:ControlMApplication` (header-row APPLICATION, `JOB_ID=1`, LEFT JOIN + `WHERE` guard).
3. Three "applications": header-row `:ControlMApplication` ≠ SEAL `:Application` ≠ folder-name `app_code`.
4. `IS_CURRENT_VERSION = '1'` is a **string**; folders use `USER_DAILY IS NOT NULL`.
5. Folder name = `SCHED_TABLE` (truth) vs `PARENT_TABLE` (denormalized on job).
6. `JOB_ID` is **folder-scoped** → identity `(folder_id, job_id)`.
7. Folder *type* = prefix position 6 (`G`), not the `DLY`/`CYC` suffix.
8. `--folder` is a **bind**, prefix `LIKE`; `%` matches DLY *and* CYC.
9. Dependencies pass **creates no nodes** — MATCHes both jobs; a missing endpoint surfaces, no ghost node.

OPEN QUESTIONS / FOLLOW-UPS